

Altimertik Interview Preparation - L2

Thursday, October 1, 2026 8:05 AM

Question-1 (Medium)

You're building a customer support bot for a fintech company. A poorly written prompt is producing inconsistent, sometimes overly casual, and occasionally incorrect responses. The current prompt is just:

"You are a helpful assistant. Answer customer question."

How would you redesign this prompt, and what specific elements would you add? Write the improved prompt.

ML/AI concepts: prompt engineering

Element	Purpose
Role	Establishes the assistant's responsibility
Context	Defines the fintech / customer-support environment
Tone	Prevent overly casual or inconsistent responses
Scope	Defines what the bot should handle
Grounding	Reduces hallucination
Uncertainty handling	Prevents the model from guessing
Safety constraints	Important for financial applications
Response format	Makes answer consistent
Escalation rules	Handles cases the AI shouldn't resolve
Examples	Demonstrates the expected behavior
Priority rules	Helps resolve competing instructions

Question-2 (Hard)

You are given a trained PyTorch classification model that perform well offline.
You now need to deploy it as an inference service and ensure correctness, performance, and reproducibility

Python: Python ML Frameworks

Ans: How I would approach it

I would divide the deployment into 6 stages:

Trained PyTorch Model

↓

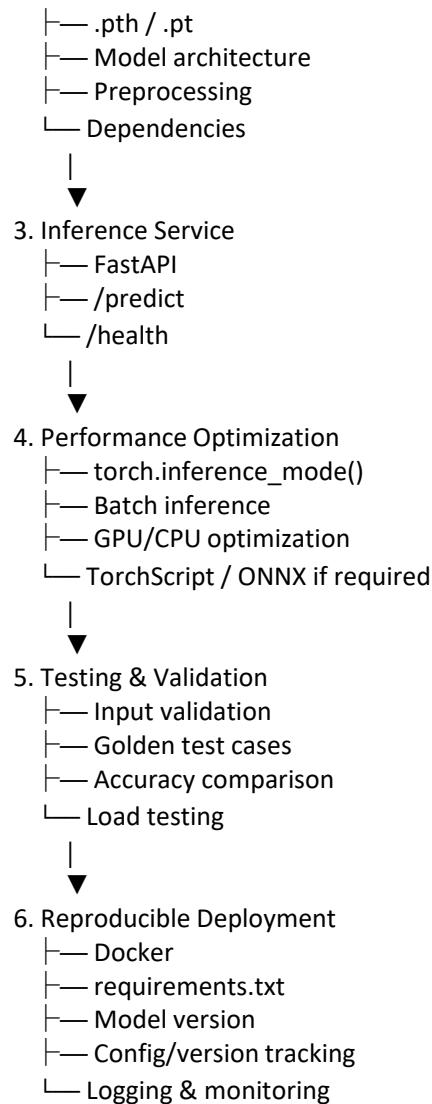


1. Model Validation
 - ├— Offline metrics
 - ├— Test predictions
 - └— Edge cases

↓



2. Model Packaging



Correctness:

The first thing I would not do is immediately expose the model through an API.

I would first verify that the deployed model produces the same results

Important Checks:

Model loading

```
model = MyClassifier(...)
model.load_state_dict(torch.load("model.pth"))
model.eval()
```

Use:

```
model.eval()
```

Because layers such as **Dropout** and **BatchNorm** behave differently during **training and inference**.

For Inference:

```
With torch.inference_mode():
    output = model(input_tensor)
```

`Torch.inference_mode()` **prevent unnecessary autograd tracking** and is preferable for inference

workloads.

Preprocessing must be identical:

A common production bug is:

Training: Resize -> Normalize -> tensor

Production: Resize -> different normalize -> tensor

The model may technically be correct, but prediction become wrong.

So I would version and package:

- Image/text preprocessing
- Normalization values
- Tokenizer if applicable
- Input dimensions
- Class mapping
- Model weights
- Model architecture
- Inference configuration

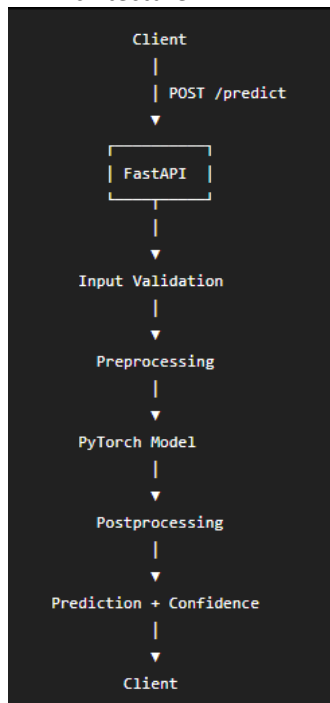
Ex: 0->cat, 1->dog, 2->horse

The class mapping must remain consistent between training and production.

Build an inference API:

Python-based deployment, I would typically use FastAPI

Ex: Architecture



Code: simplified implementation could look like

```
from fastapi import FastAPI
import torch
```

```
app = FastAPI()
```

```

model = MyClassifier()
model.load_state_dict(torch.load("model.pth", map_location="cpu"))
model.eval()

```

```

@app.get("/health")
def health():
    return {"status": "healthy"}

```

```

@app.post("/predict")
def predict(request: PredictionRequest):

    tensor = preprocess(request.input)

    with torch.inference_mode():
        output = model(tensor)

    prediction = torch.argmax(output, dim=1)

    return {
        "class_id": int(prediction.item())
    }

```

In production, I would also add proper exception handling, schema validation, structured logging, authentication/authorization where required, and request IDs.

Performance:

Once correctness is established, I would optimize performance.

First optimization

Use:

```
model.eval()
```

```

with torch.inference_mode():
    output = model(x)

```

Other techniques

Depending on the workload:

Technique	Benefit
Torch.inference_mode()	Removes inference-time autograd overhead
Batch inference	Better hardware utilization
GPU inference	Lower latency/higher throughput for suitable models
Mixed precision	Faster/lower memory usage on supported hardware
Model quantization	Lower memory and potentially faster CPU inference
TorchScript	Possible deployment optimization
ONNX	Portable optimized inference
TensorRT	High-performance NVIDIA inference
Connection pooling	Better service efficiency
Async/request batching	Better throughput for high traffic

The important interview point is:

I would optimize only after measuring the baseline.

Correctness testing:

I would create a golden dataset.

Ex:

Input A → Expected class = 2

Input B → Expected class = 0

Input C → Expected class = 1

Then compare:

Offline Model Prediction

VS

Production API Prediction

They should match within an acceptable tolerance.

For classification, I would validate

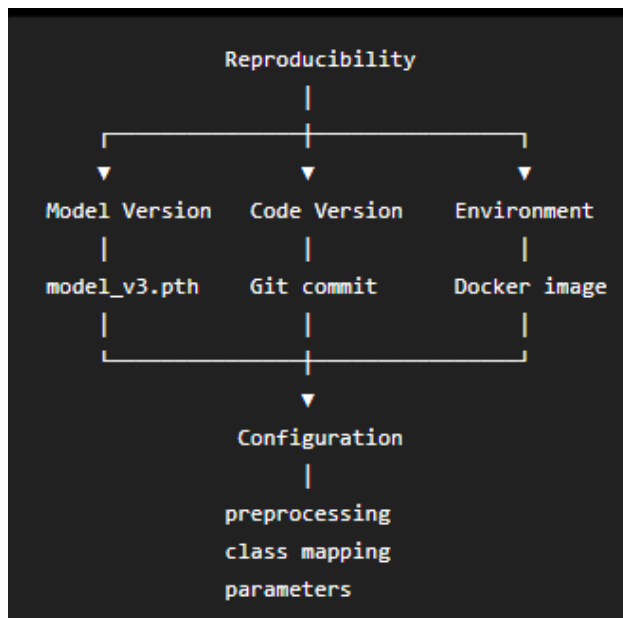
- Accuracy
- Precision
- Recall
- F1-Score
- Confusion matrix
- Per-class performance
- Confidence distribution

Edge cases:

- Empty input
- Invalid shape
- Corrupted input
- Very large input
- Unknown class
- Missing fields

Reproducibility:

I would version **everything required to reproduce the inference environment.**



I would maintain:

```
model/
  model_v3.pth
  model_metadata.json

src/
  model.py
  preprocessing.py
  inference.py
  api.py

tests/
  test_inference.py
  golden_inputs/

requirements.txt
Dockerfile
config.yaml
```

Metadata could maintain:

```

</> JSON

{
  "model_name": "customer_classifier",
  "model_version": "3.0.0",
  "framework": "pytorch",
  "pytorch_version": "2.x",
  "input_size": [224, 224],
  "classes": ["cat", "dog", "horse"],
  "preprocessing_version": "1.2.0"
}

```

Dockerization:

Docker Image: Python, PyTorch, Application code, Dependencies, Model, Configuration.

Monitoring in production:

System metrics: CPU/GPU utilization, Memory, Request throughput, P50/P95/P99 latency, Error rate

ML metrics: Prediction distribution, Confidence distribution, Class distribution, Data drift, Model performance when ground truth becomes available.

Why do we use `model.eval()` during inference?

`model.eval()` is used in **PyTorch during inference** to switch the model from **training mode** → **evaluation mode**.

Some **neural-network layers behave differently during training and inference**.

Which layers are affected?

1. Dropout

During training:

Neurons

A	B	C	D	E
↓	↓	×	↓	×

Randomly deactivate neurons

Dropout intentionally turns off random neurons to reduce overfitting.

During inference, we want **all neurons active**:

A	B	C	D	E
↓	↓	↓	↓	↓

All active

`model.eval()` disables Dropout's random behavior.

2. Batch Normalization:

BatchNorm behaves differently:

Training:

- Uses statistics from the current mini-batch.
- Updates running mean/variance.

Inference:

- Uses the learned running mean/variance from training.

`model.eval()` tells BatchNorm to use those stored statistics.

Difference between `torch.no_grad()` and `torch.inference_mode()`?

```
torch.no_grad()
|
├─ No gradient computation
└─ Some autograd bookkeeping remains

torch.inference_mode()
|
├─ No gradient computation
├─ Less autograd bookkeeping
├─ No view tracking
└─ No version-counter updates
```

therefore, for a pure inference service, `inference_mode()` can be more efficient.

How would you reduce inference latency?

1. First measure the latency

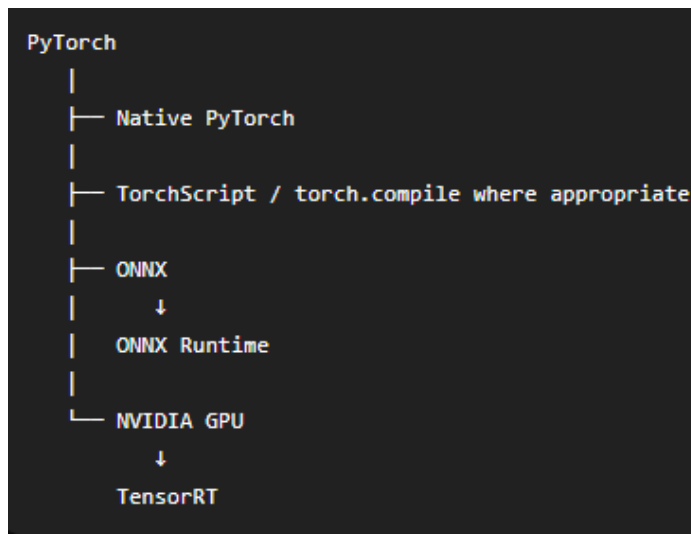
Measure: (average latency, p50, p95, p99, throughput, cpu/gpu utilization, memory usage)

For example:

```
Total latency = 100 ms

API      → 10 ms
Preprocess → 20 ms
Model    → 65 ms ← bottleneck
Postprocess → 5 ms
```

2. Use inference mode
3. Use GPU "cuda"
4. Use batching
5. Use mixed precision (FP16/BF16)
6. Optimize the model (FP32 - INT8)
7. Use optimized inference runtime



8. Optimize preprocessing (image decoding, Resizing, Tokenization, Serialization/Deserialization, CPU-GPU transfers, unnecessary data copies)
9. Optimize the API/service layer (connection pooling, Keep-alive connections, Efficient serialization, Request batching, Worker configuration, Avoiding unnecessary middleware, Async I/O where appropriate, Model warm-up)
10. Model serving architecture
11. Latency vs throughput:
 How long does one request take? Latency (Request -> prediction = 50ms)
 How many requests can the system process per second? 1000 requests / sec

Q4. Improving LLM Response Quality without fine-tuning

Sno	Technique	What it does	When to use	Key benefits
1	Prompt Engineering	Gives clear role, task, context, constraints, and instructions	Poor instruction following / inconsistent style	Better and more consistent
2	Few-shot prompting	Provides example of desired input - output behavior	Model needs to follow a specific format/style	Improves instruction
3	RAG	Retrieves relevant external knowledge and gives it to the	Domain specific or frequently changing	Reduce
4	Better Chunking	Splits docs into meaningful	Poor retrieval / context	Improves retrieved context
5	Better Embedding	Represent queries/docs semantically	Poor semantic search	Improves retrieved context
6	Hybrid	Combines keyword + semantic search	Exact terms and semantic meaning both matter	Better recall
7	Reranking	Reorders retrieved documents by relevance	Too many/irrelevant retrieved chunks	Better context
8	Context Management	Provides only relevant information to the LLM	Large/noisy context	Reduces confusion and token usage
9	Tool Calling	Allows LLM to access APIs/databases/tools	Real-time or transactional data	More accurate, current information
10	Structured Output	Forces predictable response	API/agent/data extraction use cases	Consistent machine readable output
11	Guardrails	Validates input/output and restricts unsafe behavior	Safety, compliance, hallucination	Safer responses

12	Temperature Turning	Controls response	Need deterministic vs creative output	Better consistency
13	Model Selection	Uses the right LLM for the	Current model isn't	Better quality/cost trade-off
14	Evaluation	Measure response quality systematically	Need objective quality measurement	Identifies
15	Human Feedback	Uses real user/agent	Production improvement	Finds real-world failure cases
16	Monitoring	Tracks quality and failure	Production systems	Continuous improvement

Q5: When would you choose RAG over fine-tuning?

Aspect	RAG	Fine-Tuning
Full form	Retrieval-Augmented Generation	Fine-Tuning
Changes model weights?	✗ No	✓ Yes
Adds external knowledge?	✓ Yes	⚠ Can, but not ideal for frequently changing knowledge
Good for current information?	✓ Excellent	✗ Poor choice
Updates knowledge easily?	✓ Update documents/index	✗ Usually retrain/fine-tune again
Domain-specific behavior	✓ Somewhat	✓ Excellent
Style/format adaptation	✓ Possible	✓ Excellent
Hallucination reduction	✓ Strong when well-grounded	✗ Not guaranteed
Traceability/citations	✓ Easy	✗ Difficult
Training required	✗ No	✓ Yes
Infrastructure complexity	Retrieval infrastructure	Training/fine-tuning infrastructure
Cost of updates	Low	Higher
Best use case	Knowledge grounding ↓	Behavior/task specialization

System Design question for 50k pdf should be in system with chat interface.

How will you design it?

Requirements

Functional

- User uploads PDFs (50k + documents)
- Chat interface
- Ask question across all PDFs
- Cite sources
- Support filters (Department, project, owner, date)
- Follow-up questions
- Multi-user

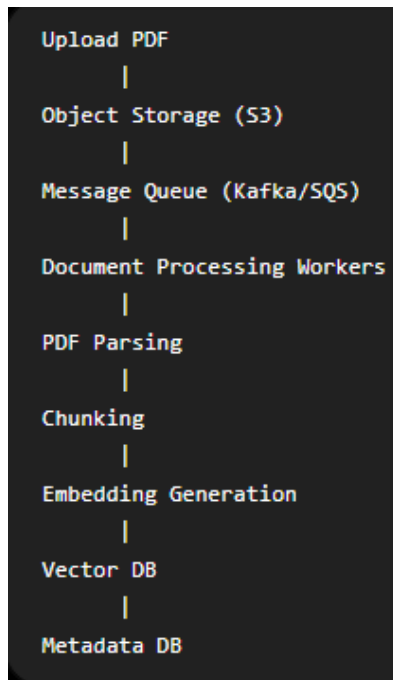
Non Functional

- < 2-3 sec response
- Highly scalable
- Fault tolerant
- Incremental indexing
- Secure (RBAC)
- High availability

Document Ingestion Pipeline

Uploading PDFs should Not directly insert into vector DB.

Instead use asynchronous processing



Step 1: Store PDF - S3, Azure Blob, GCS

Step 2: Parsing (Extract: Text, Tables, Headings, page numbers), (Libraries: PyMuPDF, PDFPlumber, Unstructure, Tika)

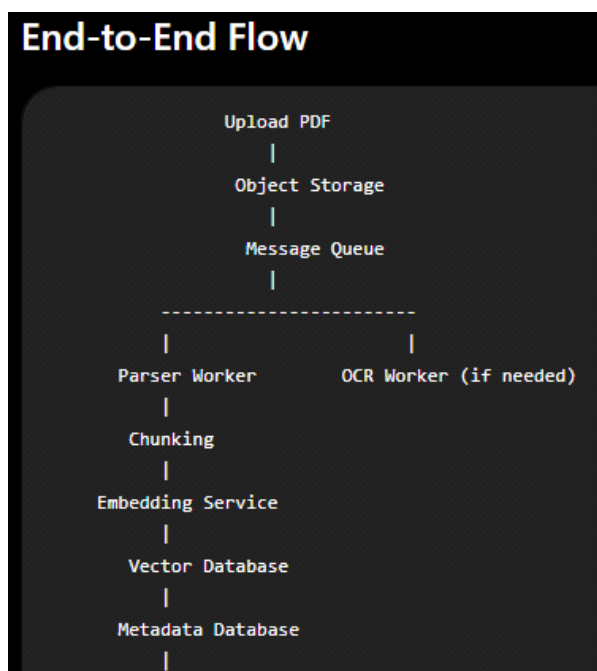
Step3: Chuncking

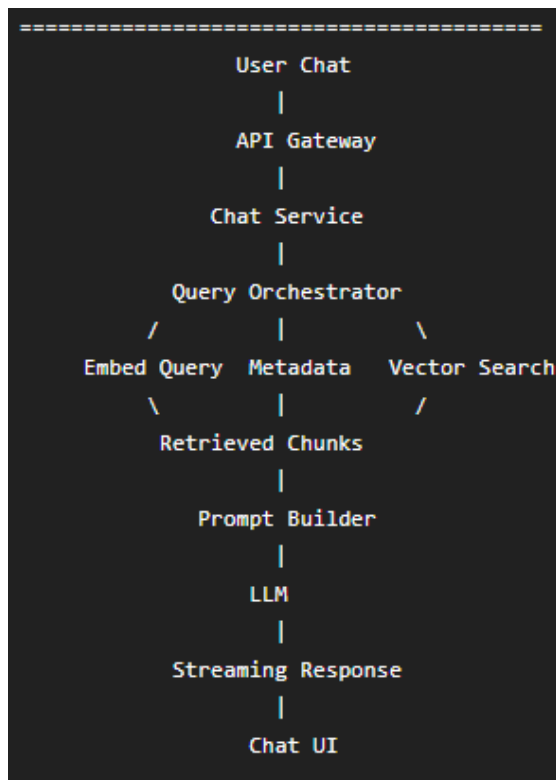
Step4: Embedding

Step5: Vector DB Store

Components:

- **API Gateway** - Authentication, Rate limiting, Routing, JWT validation
- **Query Orchestrator** - Query embedding, Metadata filters, Vector search, Prompt building, LLM call, Streaming response.
- **Metadata Database** - (Pdf, Users, Permissions, Conversation, Chunk metadata) to store in Postgres.
- **Vector DB** - (Milvus, Pinecone, Weaviate, Qdrant, pgvector (smaller deployments) used for store embeddings.
- **LLM** - GPT, Claude, Gemini, Llama
- **Scaling** - Autoscale workers
- **Queue** - Upload-> Queue -> Workers -> Index
- **Caching** - Redis (Q->A)
- **Streaming Response** - SSE (Server Sent Events or WebSockets
- **Conversation Memory**
- **Security**
- **Failure Handling** - Emb service down? (Retry via queue), LLM down (Retry or fallback model), Vector DB unavailable? (Return graceful error and log)
- **Monitoring** - Query latency (P50/P95/P99), Embedding latency, Vector search latency, LLM latency, Failed document ingestions, Token usage, Cache hit ratio.

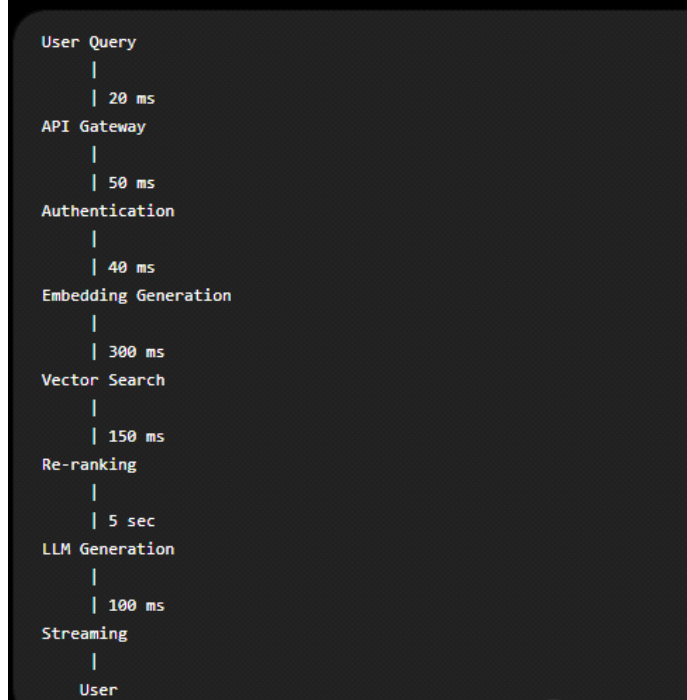




The chat response is taking 8–10 seconds. How would you identify and improve performance?

Step 1: Break down the request latency

Instead of saying "the system is slow," measure each component.



Many PDFs contain repetitive content like headers, footers, company logos,

page numbers, and disclaimers. How would you handle duplicates?

Client Discussion:

What is Temperature in an LLM?

Temperature is a parameter that controls how random or deterministic the LLM's next-token selection is.

Think of the LLM as having several possible next words:

"The capital of France is __"

The model may assign probabilities:

Token	Probability
Paris	95%
London	2%
Lyon	1%
Berlin	0.5%
Other	1.5%

Temperature = 0

The model strongly prefers the highest-probability token.

Temperature = 0

- Mostly deterministic
- More consistent
- Good for factual / structured tasks

Temperature = 0.7

The model considers lower-probability alternatives more often.

Temperature = 0.7

- More variation
- More creative
- Still reasonably controlled

Temperature = 1.5

The probability distribution becomes much flatter.

Temperature = 1.5

- High randomness
- More diverse
- Higher chance of strange/incorrect output

A simplified mathematical representation is:

$$P_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

where:

- z_i = model's original logit for token i
- T = temperature
- P_i = final probability

Lower T → sharper distribution

Higher T → flatter distribution

First, imagine this

The LLM needs to complete:

"The sun rises in the ____."

Suppose the model's original probabilities are:

east	98%
west	5%
sky	3%
morning	2%

Temperature changes the shape of this probability distribution.

1. Lower temperature → sharper distribution

Suppose:

Temperature = 0.2

The probabilities become more concentrated around the most likely choice:

east		~99%
west		~0.5%
sky		~0.3%
morning		~0.2%

So the model strongly prefers:

east

Purpose

You use a lower temperature when you want:

- consistency
- predictable answers
- less randomness
- more deterministic behavior
- structured output
- factual RAG answers
- classification
- data extraction
- code/SQL generation



2. Higher temperature → flatter distribution

Now suppose:

Temperature = 1.5

The probabilities become more spread out:

east		~65%
west		~15%
sky		~12%
morning		~8%

Now the model has more opportunity to select alternatives.

Purpose

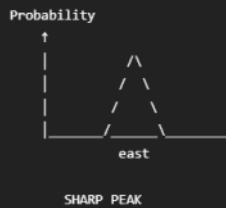
You use higher temperature when you want:

- creativity
- diversity
- brainstorming
- different wording
- storytelling
- multiple ideas
- less repetitive responses

3. Why do we call it "sharper"?

Imagine a mountain:

Low temperature



One token dominates.



High temperature

Probability



FLATTER

Several tokens become more competitive.

That's why we say:

Lower $T \rightarrow$ sharper distribution

and

Higher $T \rightarrow$ flatter distribution

How do you control temperature?

Usually through the LLM API:

Python

```
llm = ChatOpenAI(  
    model="gpt-...",  
    temperature=0.2  
)
```

For a creative application:

Python

```
llm = ChatOpenAI(  
    model="gpt-...",  
    temperature=0.8  
)
```

For structured extraction:

Python

```
llm = ChatOpenAI(  
    model="gpt-...",  
    temperature=0  
)
```

The exact supported range and behavior depend on the model/API.

What temperature should I use?

There isn't one universal value.

Use case	Typical approach
SQL generation	Low
JSON extraction	Low
Classification	Low
RAG factual Q&A	Low
Enterprise chatbot	Low-medium
Summarization	Low-medium
Code generation	Low-medium
Brainstorming	Medium-high
Story writing	High
Creative content	High

For your **RAG applications**, I'd generally start around:

Temperature = 0 → 0.3

For example:

```
</> Python
llm = ChatOpenAI(
    model="your-model",
    temperature=0.1
)
```

But temperature alone does not guarantee consistency.

Important: Temperature is NOT the only technique

This is particularly important for your interview preparation.

If an interviewer asks:

"How do you control LLM randomness and make responses consistent?"

Don't answer only:

"I set temperature to 0."

A stronger answer is:

"Temperature controls sampling randomness, but for consistent production responses I combine low temperature with deterministic decoding where supported, structured output/schema constraints, constrained prompts, retrieval grounding, fixed retrieval configuration, seed where the model supports it, and evaluation/regression testing."

Alternative techniques

1. Top-K sampling

Instead of allowing the model to select from **all possible tokens**, Top-K limits selection to the K most probable tokens.

Example:

Top-K = 5



Only the top 5 candidate tokens are considered.

Token A → 58%
Token B → 25%
Token C → 15%
Token D → 7%
Token E → 3%



Other tokens → removed

Useful for controlling randomness.

2. Top-P / Nucleus Sampling

Instead of saying:

"Consider top 10 tokens"

you say:

"Consider tokens until their cumulative probability reaches P."

Example:

Top-P = 0.9

The model considers the smallest group of tokens whose cumulative probability is approximately 90%.

This dynamically changes the candidate pool.

Difference

Temperature
↓
Changes probability distribution

Top-K
↓
Limits number of candidate tokens

Top-P
↓
Limits cumulative probability of candidates

3. Seed

Some model APIs support a **seed** parameter.

Example conceptually:

```
</> Python  
  
response = llm.generate(  
    prompt,  
    temperature=0,  
    seed=42  
)
```

The seed initializes the random process.

Same:

```
Prompt  
+ Model  
+ Parameters  
+ Seed
```

can make outputs more reproducible.

However, **seed does not necessarily guarantee exact reproducibility across every model/API/version/hardware configuration.**

4. Structured Output

This is extremely useful in production.

Instead of asking:

```
Extract customer information.
```

use a schema:

```
</> Python
```

```
class Customer(BaseModel):  
    name: str  
    age: int  
    email: str
```

Then instruct the LLM to produce that structure.

Output:

```
</> JSON
```

```
{  
  "name": "John",  
  "age": 30,  
  "email": "john@example.com"  
}
```

This reduces variation in **format**.

5. Constrained decoding

You can constrain what tokens/outputs are allowed.

For example:

Allowed output:

YES

NO

Instead of allowing the model to generate arbitrary text.

This is particularly useful for:

- classification
- JSON
- function/tool calling
- SQL generation
- controlled agents

Absolutely. **Constrained decoding** sounds complicated, but the basic idea is very simple:

Instead of allowing the LLM to say anything it wants, we put a fence around what it is allowed to generate.

Layman example

Imagine you ask a waiter:

"What is your food preference?"

Without constraints, the waiter could answer:

"I like biryani, pizza, maybe dosa, sometimes Chinese food..."

With a constraint:

You can answer ONLY **VEG** or **NON_VEG**.

Now the possible outputs are restricted:

VEG
NON_VEG

The waiter **cannot** legally choose:

PIZZA
BIRYANI
DOSA
I DON'T KNOW

That's the basic idea of **constrained decoding**.

1. What normally happens inside an LLM?

Suppose we ask:

"Is this email spam?"

The LLM predicts the next token from a huge vocabulary.

Conceptually:

SPAM	→ 45%
NOT	→ 35%
MAYBE	→ 8%
YES	→ 5%
NO	→ 4%
OTHER	→ 3%

Normally the model can potentially select from many tokens.

With constrained decoding, we say:

Only `SPAM` or `NOT_SPAM` are allowed.

So the decoder effectively works with:

SPAM	→ allowed
NOT_SPAM	→ allowed
MAYBE	→ blocked
YES	→ blocked
NO	→ blocked
OTHER	→ blocked

The model's generation process is therefore **restricted to valid outputs**.

2. Think of it as a security guard 🧑

Imagine 1,000 people want to enter a building.

Normally:

```
1,000 people
  ↓
Everyone can enter
```

Constrained decoding:

```
1,000 people
  ↓
Security guard
  ↓
Only people on the approved list
  ↓
Allowed people enter
```

For an LLM:

```
LLM vocabulary
  ↓
Constraint
  ↓
Allowed tokens
  ↓
Generated output
```

4. JSON is another great example

Suppose you ask an LLM:

Extract customer information.

Without constraints, it might return:

```
The customer is John and he is 30 years old.  
His email is john@gmail.com.
```

But your application expects:

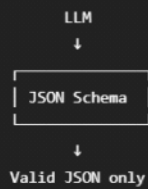
```
</> JSON  
  
{  
  "name": "John",  
  "age": 30,  
  "email": "john@gmail.com"  
}
```

You can impose a schema:

```
name → string  
age → integer  
email → string
```

Now the model has a **structure it must follow**.

Conceptually:



This is why **structured output / JSON schema** is closely related to constrained decoding.

7. Another easy example: SQL

Imagine your application wants the LLM to generate SQL.

Without constraints, the model might generate:

`</> SQL`

```
SELECT * FROM employees WHERE department = 'AI';
```

But it might also produce something invalid:

`</> SQL`

```
SELECT employees department WHERE...
```

With grammar/schema constraints, you can restrict generation to a valid SQL grammar.

Conceptually:

```
SELECT
  ↓
column
  ↓
FROM
  ↓
table
  ↓
WHERE
  ↓
condition
```

The model can't arbitrarily jump to:

banana

because `banana` isn't valid at that position in the grammar.

Tool calling

```
User
  ↓
LLM
  ↓
Tool selection
  ↓
Arguments constrained by schema
  ↓
Tool
```

Agent systems

```
Agent
  ↓
Choose action
  ↓
Search / Calculator / Database / API
```

You don't want the agent to invent arbitrary tool names or malformed arguments.

6. Prompt engineering

You can reduce variation through explicit instructions.

Instead of:

Answer the question.

use:

Answer only using the provided context.

Rules:

1. Do not use outside knowledge.
2. If the answer isn't in the context, say "Not found".
3. Return the answer in 3 bullet points.
4. Do not speculate.

This is especially important for your **RAG systems**.

7. RAG grounding

Temperature doesn't solve hallucination by itself.

For example:

```
User question
  ↓
Retriever
  ↓
Relevant documents
  ↓
LLM
  ↓
Answer
```

You can tell the LLM:

```
Answer ONLY from the retrieved context.
```

This reduces the model's freedom to invent information.

8. Retrieval configuration

For a RAG system, consistency also depends on retrieval.

For example:

```
</> Python
retriever = vectorstore.as_retriever(
    search_kwargs={"k": 5}
)
```

If retrieval changes dramatically between requests, the final answer can change even when:

```
temperature = 0
```

So you should control:

- chunking
- embedding model
- similarity metric
- `top_k`
- metadata filters
- reranking
- score threshold
- query rewriting

9. Reranking

A common production RAG architecture is:

```
Query
↓
Vector Search
↓
Top 20 documents
↓
Reranker
↓
Top 5 documents
↓
LLM
```

This gives the LLM a more consistent and relevant context.

10. Output validation

You can validate the LLM's answer after generation.

Example:

```
LLM
↓
JSON validation
↓
Schema validation
↓
Business-rule validation
↓
Accept / Retry
```

For example:

```
</> Python

if not valid_response:
    retry()
```



This is much stronger than relying only on temperature.

11. LLM-as-judge / evaluation

For production applications, measure consistency rather than assuming it.

For example:

```
Same question
↓
Run 10 times
↓
Compare answers
↓
Measure:
- factual consistency
- semantic similarity
- JSON validity
- citation correctness
```

You can use evaluation frameworks such as RAGAS or DeepEval depending on the application.

System design questions for a 50k PDF should be in a system with a chat interface. How will you design it?

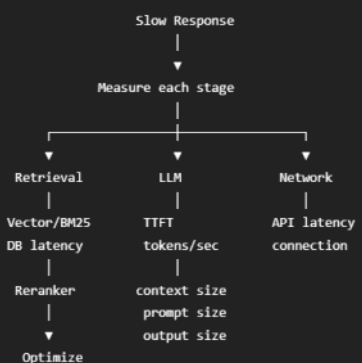
End-to-End Flow



What are the performance you will focus if the response is slow?

11. My debugging order in an interview

I'd answer this very systematically:



2. Most important metrics

I would monitor:

Metric	What it tells me
TTFT	Time until first LLM token
Total latency	Complete response time
Tokens/sec	LLM generation speed
Embedding latency	Query embedding bottleneck
Retrieval latency	Vector/BM25 search performance
Reranker latency	Whether reranking is expensive
Context size	Too much information sent to LLM
Input tokens	Prompt processing cost/latency
Output tokens	Generation time
Cache hit rate	Whether caching is helping
CPU/GPU utilization	Infrastructure bottleneck
Concurrent requests	Load/scaling problem

How you will handle the duplicate repetitive format in pdf?

1. First identify what kind of duplicate it is

There are usually four cases:

```
PDFs
|
├─ Exact duplicate PDF
|
├─ Same PDF with different filename
|
├─ Repeated boilerplate inside documents
|   └─ Header / footer / disclaimer
|
└─ Near-duplicate documents
    └─ Version 1, Version 2, Version 3
```

Each needs a different approach.

2. Exact duplicate PDFs

Before parsing, calculate a file hash.

```
PDF
↓
SHA-256
↓
Hash
```

Example:

```
manual_A.pdf → abc123
manual_B.pdf → abc123
manual_C.pdf → xyz789
```

If:

```
hash(A) == hash(B)
```

then they are byte-for-byte identical.

I can store:

```
document_hash
```

in the metadata/database and avoid processing the same file again.

3. Same content but different filename

For example:

```
Radar_Manual.pdf  
Radar_Manual_Final.pdf  
Radar_Manual_Copy.pdf
```

The file hashes may be different because PDF metadata or internal structure changed.

So after extraction:

```
PDF  
↓  
Extract normalized text  
↓  
Normalize  
↓  
Content fingerprint
```

Normalization could include:

```
lowercase  
remove extra whitespace  
remove repeated headers  
remove page numbers  
normalize punctuation
```

Then generate a fingerprint/hash.

```
Normalized content  
↓  
SHA-256  
↓  
Content hash
```

Now you can detect documents that are essentially identical.

4. Repeated headers and footers

This is very common in PDFs.

Imagine every page contains:

```
DEFENCE RADAR SYSTEM — CONFIDENTIAL
```

and:

```
Page 1  
Page 2  
Page 3  
...
```

If you don't remove this, your chunks become:

```
Chunk 1:  
DEFENCE RADAR SYSTEM  
Page 1  
Actual content...  
  
Chunk 2:  
DEFENCE RADAR SYSTEM  
Page 2  
Actual content...  
  
Chunk 3:  
DEFENCE RADAR SYSTEM  
Page 3  
Actual content...
```

Now the embedding model sees the same information repeatedly.

Solution

During parsing, identify repeated text appearing at the same location across many pages.

For example:

```
Page 1 → "DEFENCE RADAR SYSTEM"  
Page 2 → "DEFENCE RADAR SYSTEM"  
Page 3 → "DEFENCE RADAR SYSTEM"  
...  
Page 100 → "DEFENCE RADAR SYSTEM"
```

If it appears on, say, **80%+ of pages**, classify it as a repeated header/footer candidate.



Then remove it before chunking.

5. Boilerplate paragraphs

Sometimes the repetition isn't just headers.

Example:

```
This document contains proprietary information...  
This document contains proprietary information...
```

across hundreds of documents.

I would maintain a **boilerplate detection layer**.

```
All documents  
↓  
Sentence extraction  
↓  
Frequency analysis  
↓  
Repeated sentences/paragraphs  
↓  
Boilerplate candidates
```

For example:

```
Sentence A → appears in 45,000 PDFs  
Sentence B → appears in 32,000 PDFs  
Sentence C → appears in 2 PDFs
```

A and B are likely boilerplate.

But I would be careful about automatically deleting them. Some repeated text may actually contain important domain information.

So I'd distinguish:

```
Boilerplate  
↓  
Remove / downweight  
  
Important repeated statement  
↓  
Keep
```

6. Near-duplicate chunks

This is particularly important.

Suppose:

```
Chunk A:  
Radar systems detect airborne targets using...  
  
Chunk B:  
Radar systems are used to detect airborne targets using...
```

They're not exactly identical, but semantically almost the same.

For this, I can use:

MinHash / SimHash

Good for detecting textual similarity efficiently.

Or:

Embedding similarity

```
Chunk A → vector A  
Chunk B → vector B  
  
cosine_similarity(A, B) = 0.97
```

If similarity exceeds an experimentally validated threshold, mark them as near duplicates.

For millions of chunks, I wouldn't perform an expensive **all-pairs comparison**:

```
5 million × 5 million ❌
```

Instead, use techniques such as:

```
MinHash / LSH  
ANN search  
document-level clustering
```

to make duplicate detection scalable.

7. Versioned documents are different

This is an important interview point.

Suppose I have:

```
Radar_Manual_v1.pdf
Radar_Manual_v2.pdf
Radar_Manual_v3.pdf
```

I shouldn't simply delete v1 and v2.

Maybe the user asks:

What changed between v1 and v3?

Therefore I would maintain document lineage:

```
Radar_Manual
├─ v1
├─ v2
└─ v3
```

Metadata:

```
</> JSON
{
  "document_id": "RADAR001",
  "version": "3",
  "status": "active",
  "parent_version": "RADAR001_V2"
}
```

For normal QA:

```
status = active
```

can be used as a retrieval filter.

For historical queries:

```
include archived versions
```

can be enabled.

8. Duplicate retrieval results

Even after deduplication, retrieval itself can return repetitive chunks.

Example:

Top 10

1. Radar manual page 20
2. Radar manual page 21
3. Radar manual page 22
4. Radar manual page 23
5. Radar manual page 24

All five may contain almost the same paragraph.

So I would apply **diversification after retrieval**.

One approach is **MMR — Maximal Marginal Relevance**.

Conceptually:

Relevance to query
+
Diversity from already-selected chunks

Instead of:

Top 5 = almost identical

we want:

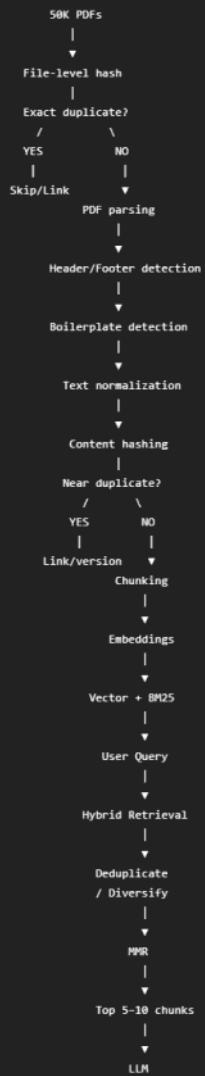
Top 5

1. Definition
2. Architecture
3. Processing method
4. Example
5. Limitation

This gives the LLM more useful information.

9. The complete pipeline

For my 50K-PDF system, I would implement:



what if user search a short key word in the chat how rag will handle?

This is a **very important RAG interview question**. Short queries like:

radar
BERT
Doppler
AWS
HViT

can be difficult for pure semantic search because there isn't enough context.

The solution is **not to depend only on vector search**. I would use **query-type detection + hybrid retrieval + query expansion**.

1. Problem with pure vector search

Suppose the user enters:

HViT

The embedding model converts it into a vector and searches semantically.

But the user may actually want documents containing the **exact term** **HViT**.

A semantic search might return:

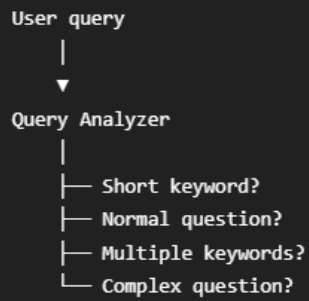
Vision Transformer
Transformer architecture
Object detection
Attention mechanism

while the exact documents containing **HViT** may rank lower.

Therefore, for short keywords, **BM25/keyword search becomes extremely important**.

2. Detect whether the query is short

I would have a lightweight query-understanding layer:



For example:

"radar"

→ short keyword

"How does radar detect aircraft?"

→ natural-language question

"radar Doppler clutter"

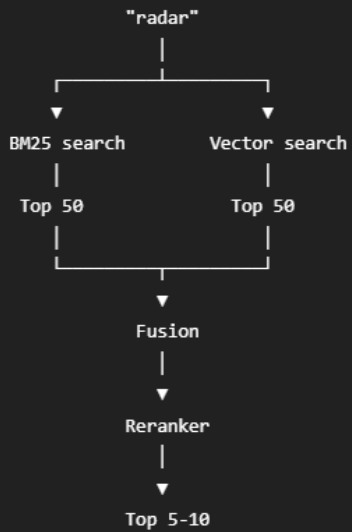
→ multiple keywords

3. Short keyword → Hybrid search

For:

radar

I would increase the importance of exact/lexical retrieval:



BM25 can find:

Radar System
Radar Detection
Radar Signal Processing
Radar Architecture

even though the query is only one word.

4. Exact match should be preserved

I would also maintain an **exact-match** signal.

For example:

```
Query = "HvIT"
```

A document containing:

```
REDET + STD + HvIT
```

should receive a strong lexical score.

Conceptually:

```
Final Score =  
  α × BM25  
  + β × Vector Similarity  
  + γ × Exact Match
```

The exact weights would be tuned using evaluation data rather than hard-coded blindly.

how lexical score should be calculating the parameter alpha, beta, gamma?

Yes. This is where you move from a **conceptual RAG design** to a proper **ranking system**.

Suppose we have:

$$Score(d, q) = \alpha S_{BM25} + \beta S_{vector} + \gamma S_{exact}$$

The important point is:

You should not choose α , β , γ arbitrarily. First normalize the scores, then tune the weights using a validation dataset.

1. Why normalization is necessary

BM25 and cosine similarity have different ranges.

For example:

```
BM25 score      = 14.7  
Vector similarity = 0.82  
Exact match     = 1
```

If you simply do:

```
0.3 × BM25  
+ 0.5 × vector  
+ 0.2 × exact
```

BM25 can dominate because its numerical scale is different.

So first normalize them.

2. Normalize each score

For example, using min-max normalization:

$$S' = \frac{S - S_{min}}{S_{max} - S_{min}}$$

Then:

BM25:

14.7 → 0.91

Vector:

0.82 → 0.82

Exact:

match → 1.0

no match → 0

Now they are approximately on the same scale:

BM25 → 0-1

Vector → 0-1

Exact → 0-1

Then weighted combination makes sense.

3. What do α , β , γ mean?

For:

$$Score = \alpha BM25 + \beta Vector + \gamma Exact$$

α = lexical importance

How much should exact keyword matching matter?

Useful when the query contains:

```
HViT
AN/APG-81
F-35
ABC-123
PRI 1250
```

β = semantic importance

How much should meaning/similarity matter?

Useful for:

```
"How can radar detect moving targets?"
```

A document might not contain exactly those words but could contain:

```
| Doppler-based detection of airborne moving objects.
```

Semantic retrieval can find it.

γ = exact-match importance

This gives an explicit boost when the exact keyword appears.

For example:

```
Query = "HViT"
```

If:

```
Document A → contains "HViT"
Document B → contains "Vision Transformer"
```

you may want Document A to receive a strong boost.

4. Don't use one fixed α , β , γ for every query

This is the important part.

For your 50K-PDF system, I would use **query-dependent weighting**.

Short keyword

Query:

"HViT"

Possible starting point:

$\alpha = 0.45$

$\beta = 0.20$

$\gamma = 0.35$

Here lexical and exact matching are important.

Normal natural-language question

How does HViT improve object detection?

Possible starting point:

$\alpha = 0.30$

$\beta = 0.55$

$\gamma = 0.15$

Semantic understanding gets more importance.

These are **starting hypotheses, not universal values**.

5. How do I actually calculate the weights?

This is where evaluation comes in.

Create a representative test dataset.

For example:

Query	Relevant documents
"HVI1"	DOC12, DOC18
"radar clutter"	DOC21, DOC45
"How does Doppler work?"	DOC33
"AN/APG-81"	DOC51
"target detection methods"	DOC71, DOC72

Suppose you have:

500 test queries

For every query, you know which documents/chunks are relevant.

Then test different weight combinations.

For example:

α	β	γ
0.2	0.6	0.2
0.3	0.5	0.2
0.4	0.4	0.2
0.5	0.3	0.2
0.4	0.2	0.4

Run each configuration against your evaluation dataset.

Measure:

Recall@5
Recall@10
MRR
NDCG@10
Precision@K

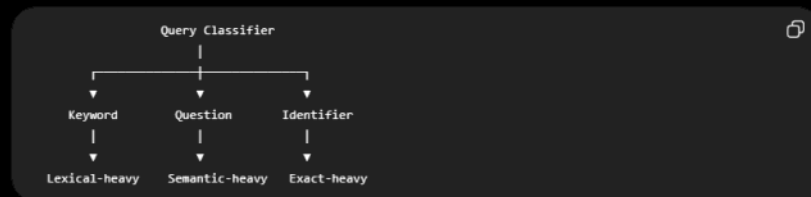
You might discover:

Configuration	Recall@10
0.2 / 0.6 / 0.2	82%
0.3 / 0.5 / 0.2	87%
0.4 / 0.4 / 0.2	91%
0.5 / 0.3 / 0.2	88%

You then select the configuration based on your **predefined retrieval objective**, rather than guessing the weights.

6. Better approach: separate query types

I would actually create different weighting profiles.



For example:

Query	α BM25	β Vector	γ Exact
radar	0.45	0.20	0.35
radar clutter suppression	0.40	0.40	0.20
How does radar detect aircraft?	0.25	0.60	0.15
AN/APG-81	0.30	0.15	0.55

Again, these are **illustrative starting values**. Your evaluation set determines the production values.

7. One more important issue: BM25 score itself

You asked specifically about **lexical score calculation**.

BM25 is roughly:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b \frac{|d|}{avgdl})}$$

Where:

- $f(t, d)$ = frequency of term t in document d
- $IDF(t)$ = inverse document frequency
- k_1 = controls term-frequency saturation
- b = controls document-length normalization
- $|d|$ = document length
- $avgdl$ = average document length

You normally **don't manually calculate BM25 in your application**.

Your search engine such as Elasticsearch/OpenSearch calculates it.

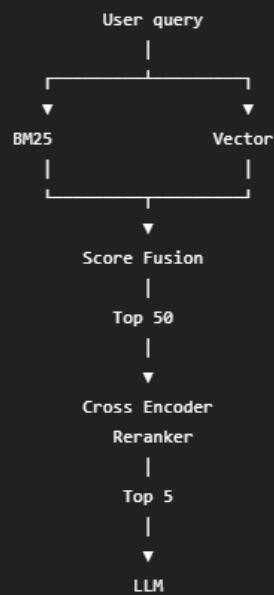
Your responsibility is primarily:

```
Query
↓
BM25
↓
raw lexical score
↓
normalize/fuse
```

9. Even better: use a reranker

In a production RAG system, I wouldn't try to make $\alpha/\beta/\gamma$ solve everything.

I'd do:



So:

$\alpha/\beta/\gamma$ = candidate retrieval/fusion

and

Reranker = more precise final relevance judgment.

5. What if the keyword has multiple meanings?

This is where query expansion helps.

Suppose the user searches:

"ARIS"

The system doesn't know whether the user means:

ARIS software
ARIS system
ARIS architecture
ARIS workflow

We can search exact matches first and inspect metadata/content.

For example:

ARIS
↓
Exact match search
↓
Documents containing ARIS
↓
Extract surrounding context

Then the system can understand the likely domain.

6. Query expansion

For short queries, I can generate additional search terms.

Example:

User:
"RAG"

Query expansion:

RAG
Retrieval Augmented Generation
retrieval-augmented generation
retrieval pipeline
vector retrieval

Then:

Original query
+
Expanded terms
↓
Hybrid retrieval

But I wouldn't automatically expand **every short keyword**, because expansion can introduce noise.

For exact identifiers such as:

HViT
F-35
AN/APG-81
XYZ-123

I would preserve the original exact term as a high-priority signal.

7. Search title, section and metadata too

Suppose:

```
User:  
"Radar"
```

I would search not only chunk text but also:

```
document title  
section title  
keywords  
metadata  
filename
```

Example:

```
Document:  
"Advanced Radar Signal Processing.pdf"  
  
Section:  
"Radar Clutter Suppression"  
  
Content:  
"...radar systems..."
```

All three provide useful signals.

So my index could contain:

```
</> JSON  
  
{  
  "title": "Advanced Radar Signal Processing",  
  "section": "Radar Clutter Suppression",  
  "text": "...",  
  "keywords": ["radar", "clutter", "Doppler"]  
}
```

8. Don't immediately send 100 results to the LLM

This is another important point.

If someone types:

radar

there may be **10,000 matching chunks**.

I would not send them to the LLM.

Instead:

```
"radar"
↓
BM25 + Vector
↓
Top 50
↓
Metadata filtering / deduplication
↓
Reranker
↓
Top 5-10
↓
LLM
```

9. But what does the user actually want?

A one-word query is sometimes not really a question.

If the user types:

radar

the best UX may be to return a **search-style result**, rather than forcing the LLM to invent an explanation.

For example:

I found information about "radar" in 326 documents.

Top relevant documents:

1. Radar Signal Processing Manual
Pages 12-28
2. Advanced Radar Architecture
Pages 4-19
3. Radar Detection Techniques
Pages 31-45

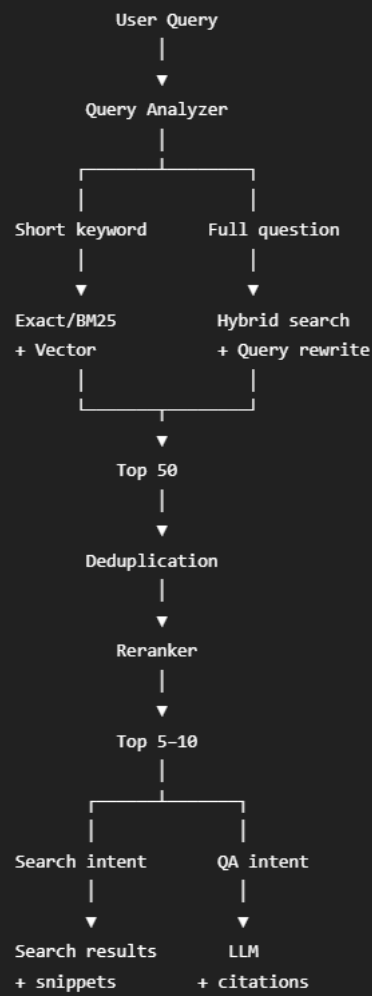
Then the user can ask:

"Explain Doppler processing."

This avoids hallucination and gives the user control.

10. Very short query decision flow

I'd design it like this:



5. Important: Don't force the chatbot to guess

For a short query like:

`Radar`

The chatbot could also ask:

"Are you looking for radar-related documents, or would you like an explanation of radar?"

This is called **clarification**.

It is useful when the user's intention is unclear.

However, you don't need to ask this for every short keyword. You can use a search-first approach and allow the user to continue asking questions.

Interview-friendly meaning

When I said:

"What does the user actually want?"

I meant:

Does the user want to search for documents, or do they want the LLM to explain something from the documents?

For your 50K-PDF chatbot:

- `Radar` → likely search/discovery.
- `What is radar?` → question answering.
- `Compare radar system A and B` → comparison.
- `Find all documents related to radar` → document search.

One more important point: **This is not a mandatory separate LLM call**. You can implement it using rules, a lightweight classifier, or an LLM depending on the complexity and latency requirements.